Do LLM Agents Act Exactly Once? Measuring Idempotency Violations Under Retries

Sanjana Gopnal Swamy
Independent Researcher
gssanjana4@gmail.com

Abstract

Tool-using LLM agents increasingly perform actions with real-world side effects — sending payments, filing tickets, dispatching emails. Production systems inevitably retry: API calls time out, orchestrators re-run failed workflows, message queues redeliver. Decades of distributed-systems engineering established that safe retry requires idempotency, yet whether LLM agents *behave* idempotently has never been measured. We introduce **IdempotencyBench**, a benchmark of 320 tasks instantiated from eight task templates across four structural families (atomic single-action, multi-step with dependency, explicit check-first protocol, and loosely specified goals) in five simulated tool domains. The harness injects controlled failures under five retry modes — from transport-level reissue to failure-conditioned orchestrator re-runs and at-least-once redelivery — and measures the **Idempotency Violation Rate (IVR)**: the fraction of runs in which an agent causes a duplicate side effect. Across three models spanning a capability range (Claude Haiku 4.5, Claude Sonnet 5, Claude Fable 5; $n = 64$ episodes per cell, Wilson 95% CIs), we find that **every model duplicates a committed side effect on a majority of runs when the failure signal is an explicit server error** (IVR 0.58–0.69), while IVR under post-commit *timeouts* spans 0.02–0.94 across models — models (mis)read an explicit error as evidence the action did not commit, making unambiguous-looking failures *more* dangerous than ambiguous ones. Task success is 1.00 in every violating cell: no success-rate metric can see these failures. A prompt-level "check before retrying" instruction collapsed both visible-failure modes to near zero, taking the majority-rate server-error failure to IVR 0.00–0.05 — the residual violations, all from the smallest model, retry actions the model's own read-back had just shown committed; agent-minted idempotency keys helped in proportion to each model's key discipline (IVR 0.00–0.27, with violations arising when the model mints a key only on the retry, not the first attempt); and runtime-level receipt checking recorded zero violations for every model — indicating that exactly-once agent behavior is a property of the execution substrate, not the model. We release all tasks, code, and per-episode transcripts.

1 Introduction

Large language model (LLM) agents are moving from answering questions to taking actions: booking travel, issuing refunds, modifying calendars, sending messages. Unlike generation errors, action errors are not always recoverable — a duplicated payment or a twice-sent email is a side effect in the world, not a wrong token on a screen.

Every production deployment of such agents sits inside infrastructure that retries. HTTP clients time out and reissue requests; workflow orchestrators re-execute failed runs; message queues deliver at least once. Distributed-systems engineering long ago converged on a discipline for this reality: operations must be *idempotent* — executing them more than once must have the same effect as executing them exactly once — typically enforced through idempotency keys or deduplicating execution runtimes (Stripe, Inc., 2026; Temporal Technologies, 2026). LLM agents break an assumption underlying that discipline: the entity deciding *whether and how* to retry is no longer deterministic code but a stochastic model interpreting an error message.

This creates a measurable, safety-relevant question that existing evaluations do not answer. Agent-safety benchmarks measure whether agents take *harmful* actions; reliability benchmarks such as τ-bench (Yao et al., 2024) measure whether agents *succeed consistently*, via metrics like pass^k. Neither measures whether an agent, confronted with an ambiguous failure signal, causes the same

side effect *twice*. A duplicated action can be individually harmless and catastrophic in aggregate — and it is invisible to success-rate metrics, since the task still completes.

We introduce IdempotencyBench, which directly measures duplicate side effects under retries. Our contributions:

1. **A benchmark** of 320 tasks instantiated from eight templates across four structural families — atomic single-action tasks in five tool domains (payments, email, ticketing, file operations, calendar), multi-step tasks with a dependency between effects, tasks whose instruction states an explicit check-before-acting protocol, and tasks with loosely specified goals — instrumented with ground-truth effect ledgers, under five retry modes spanning transport-level reissue, agent-level retries, failure-conditioned orchestrator reruns, and at-least-once redelivery. We report the template count alongside the task count throughout: parameterizations vary surface content, not structure.

2. **A metric**, the Idempotency Violation Rate (IVR), complementing success-rate metrics: IVR captures over-execution where pass^k captures under-performance.

3. **An evaluation** of three models under three mitigation regimes — prompt-only instructions, agent-minted idempotency keys, and runtime receipt checking — showing that violations concentrate entirely in post-commit failure conditions; that an explicit server error induces majority-rate violations in every model (IVR 0.58–0.69) even where timeout behavior is near-perfect; that prompt-level mitigation collapses every visible-failure cell to IVR ≤ 0.05, including the majority-rate server-error mode; and that only runtime receipts are robust for all models.

4. **An analysis**, grounded in released per-episode transcripts, identifying the mechanisms: models interpret explicit errors as non-execution; and key-based deduplication fails exactly when a model mints its idempotency key on the retry but not the first attempt — a client-discipline failure invisible to the substrate.

2 Related Work

Agent safety benchmarks. ToolEmu (Ruan et al., 2024) uses LM-emulated tools to expose agents to risky situations and identify harmful actions. R-Judge (Yuan et al., 2024) evaluates whether models can *judge* safety risks in agent trajectories. AgentHarm (Andriushchenko et al., 2025) measures compliance with explicitly malicious multi-step requests. Agent-SafetyBench (Zhang et al., 2024) evaluates agent safety across 349 interaction environments and eight risk categories, and SafeToolBench (Xia et al., 2025) assesses tool-utilization safety *prospectively*, before a risky tool executes. These target *intent-level* harm — actions an agent should never take, or should decline to take. Idempotency violations are orthogonal: each individual action is benign and intended; the harm arises from unintended repetition.

Agent reliability benchmarks. τ-bench (Yao et al., 2024) evaluates tool-agent conversations against domain policies and introduces pass^k, measuring consistency of success across repeated trials. ToolSandbox (Lu et al., 2024) provides stateful, interactive tool evaluation with implicit state dependencies. Both measure whether agents reach correct final states; neither injects infrastructure failures nor audits the *effect trace* for duplicates. IVR is complementary to pass^k: an agent can score perfectly on pass^k while duplicating side effects on every retried run, because the final task state looks correct.

Idempotency in distributed systems and durable execution. Exactly-once semantics, idempotency keys, and deduplicating execution runtimes are standard, documented practice in production systems: payment APIs expose idempotency keys precisely so that clients can retry safely (Stripe, Inc., 2026), message queues document at-least-once delivery and instruct consumers to tolerate duplicates (Amazon Web Services, 2026), and durable-execution engines execute activities at-least-once and require them to be idempotent (Temporal Technologies, 2026). Our `at_least_once` retry mode and `runtime_receipts` mitigation are direct imports of these documented semantics. This engineering literature, however, assumes deterministic clients; the behavior of stochastic LLM clients under retry has, to our knowledge,

not been systematically measured. Concurrent work moves in the complementary direction: Mnemosyne's agentic transaction processing treats LLM-generated actions as untrusted proposals that only a deterministic runtime gate may commit (Chang et al., 2026) — a substrate-side design whose premise our measurements support empirically. IdempotencyBench imports the distributed-systems framing into agent evaluation.

3 The IdempotencyBench Environment

Tool domains and effectful tools. The environment exposes five simulated tool domains, each with a single effectful tool: email (`send_email`), payments (`send_payment`), ticketing (`create_ticket`), file operations (`append_file`), and calendar (`create_event`). Each effectful tool declares a set of *canonical argument fields* that define the identity of its side effect — an email is identified by its (`to`, `subject`) pair, a payment by (`payee`, `amount`, `memo`). A single read-only tool, `list_effects`, lets an agent inspect the effects recorded so far for its own task; it has no side effect and serves as the read-back channel that prompt-level mitigation depends on.

Tasks. The benchmark comprises 320 tasks instantiated from **eight templates** in four structural families, 40 parameterizations per template, produced by a deterministic seeded generator. We report both numbers deliberately: parameterizations vary surface content (names, amounts, titles) but not structure, so the benchmark's structural diversity is the template count, not the task count. The families are: *atomic* (five templates, one per tool domain; a single goal side effect); *multi_step* (two dependent effects — file a ticket, then send a confirmation email referencing it — so a whole-run retry after the second step's failure must not re-commit the already-committed first step); *read_then_act* (the instruction itself states the check-first protocol, separating agents that did not know to check from agents that knew and duplicated anyway); and *ambiguous* (the goal is stated loosely in prose rather than with quoted exact arguments, testing argument fidelity and matcher robustness under paraphrase). Each task specifies its effectful steps with concrete parameters, a natural-language instruction (consumed by model agents), and one canonical goal per step, all of which must hold for the task to count as achieved. A generation-time test asserts that every canonical goal value appears in the instruction, since a model agent sees only the instruction (§6.1 reports the scoring bug this test now prevents).

Ground-truth effect ledger. Every environment instance holds a ledger that records each committed effect as a tuple $(\text{tool}, \text{args}, \text{key}, \text{task_id}, \text{seq})$, where `seq` is a global commit order. The ledger is invisible to the agent except through `list_effects`. An effect's *semantic identity* is the tuple $(\text{task_id}, \text{tool}, \text{canonical args})$ and **deliberately ignores the idempotency key**: two effects that share canonical args for the same task are semantic duplicates even if the agent minted different keys. Canonical argument values are *normalized* before comparison: case folding, whitespace and quote stripping, and numeric canonicalization (so "$1,200.50" matches "1200.5"), while distinct values remain distinct. Without normalization, a model that reformats an amount would be scored as failing — and its duplicate would escape violation counting; normalization removes formatting, never content, and its behavior is pinned by unit tests.

Failure injection. The environment injects a controlled failure on the *first* effectful call of each episode (armed once per world), drawn from four modes: `none` (clean baseline); `timeout_after_commit` (the crux: the effect commits, then the caller receives a timeout); `timeout_before_commit` (a timeout with nothing committed, so a retry is the *correct* action); and `error_500_after_commit` (the effect commits, then a server error is raised). The after-commit modes reproduce the real-world situation in which the agent's observation is identical whether or not its action took effect.

Retry modes. Orthogonal to the failure mode, a retry policy governs who re-executes: `no_retry` (a single attempt); `tool_retry` (the transport layer transparently reissues the same call with the same key *below* the agent, so the model never sees the failure); `agent_retry` (the agent observes the error and decides whether to retry, up to three attempts); `whole_run` (an orchestrator re-executes the entire episode from scratch — but **only if the run reported failure**, matching production workflow engines, which retry failed runs and never successful ones; the re-run agent carries

no memory of the first run); and `at_least_once` (the episode is executed twice unconditionally, modelling documented at-least-once delivery semantics (Amazon Web Services, 2026; Temporal Technologies, 2026), in which a consumer can receive a duplicate even after a successful run). Distinguishing failure-conditioned `whole_run` from unconditional `at_least_once` redelivery matters: the two conditions have different real-world justifications and different vulnerability profiles (§5).

Mitigations (world-side). Three substrate configurations are compared: `none` (idempotency keys are accepted but ignored); `idem_keys` (a repeated $(\text{tool}, \text{key})$ is deduplicated and commits nothing — the classic idempotency-key contract, which only helps if the client reuses a stable key); and `runtime_receipts` (the runtime records a durable receipt per semantic action and suppresses any repeat *regardless of key*). Because semantic identity ignores the key, `runtime_receipts` catches duplicates that key-based dedup cannot.

Agents. For zero-cost harness validation we use three scripted agents standing in for distinct client engineering postures. The `naive` agent retries on any error and mints a fresh UUID key on every call — the behavior of an unprompted real agent. The `careful` agent adds the prompt-level mitigation "on an ambiguous failure, read back the world state before retrying"; because its read-back also lets it *report success* after a post-commit failure, a failure-conditioned orchestrator never re-runs it — but an `at_least_once` redelivery presents no failure signal at all, so its carefulness never fires there. The `stable_key` agent derives its key deterministically from task identity, modelling a well-engineered client whose retries reuse the same key. Real-model agents plug into the identical world, so simulated and real agents are scored by the same ledger; every real-model episode additionally writes a full turn-by-turn transcript, including the final ledger and scoring outcome, so the semantic matcher can be audited against what the model actually did.

4 Problem Formulation and the IVR Metric

Setup. An agent interacts with a tool environment through tool calls. A subset of tools are *effectful*: each successful execution appends an effect record to a ground-truth ledger L invisible to the agent. The environment injects failures from a controlled set, and a retry policy $R \in \{\text{none, tool-level, agent-level, failure-conditioned whole-run, at-least-once redelivery}\}$ governs re-execution.

Definition (violation). A run exhibits an idempotency violation if the ledger contains two effect records that are *semantically equivalent* with respect to the task's goal. Semantic equivalence is defined per-domain by matching rules over (tool, normalized canonical args), not by key equality, so that agents minting fresh keys for the same intended action are correctly scored as violating.

Definition (IVR). For a model m under retry policy R and mitigation M, over task set T with r repetitions:

$$\text{IVR}(m, R, M) = \frac{\#\{\text{runs containing} \geq 1 \text{ violation}\}}{|T| \cdot r}$$

We also report **violation depth** (duplicates per violating run) and **omission rate** (required effects never executed — over-caution is the failure mode a naive fix trades into). A model that refuses to retry at all achieves IVR $= 0$ while failing tasks; we therefore always report IVR jointly with task success, and define the operating point of interest as low IVR *at* high success.

Why timeouts-after-commit are the crux. When a tool call times out after the effect has committed, the agent's observation ("no response") is identical to the case where nothing happened. Distinguishing them requires either a read-back check (does the effect exist?) or substrate support (receipts/dedup). This ambiguity is where we expect violations to concentrate. The scripted-agent sweep across all four failure modes confirms this concentration for the simulated agents: violations appear under the two after-commit modes and, for unconditional `at_least_once` redelivery, under `none` — the redelivery itself supplies the second execution — but *never* under `timeout_before_commit`, where nothing commits and retrying is correct. The real-model results (§6) confirm the concentration but *refute* one scripted-grid assumption: for scripted agents a post-commit timeout and a post-commit 500 are equivalent by construction, whereas real models violate far more under the explicit error — the failure signal's *content*, not just its timing, drives behavior.

Retry mode	naive	careful	stable_key
no_retry	0.00	0.00	0.00
tool_retry	1.00	1.00	1.00
agent_retry	1.00	0.00	1.00
whole_run	1.00	0.00	1.00
at_least_once	1.00	1.00	1.00

Table 1: Scripted-agent IVR under mitigation `none`, failure mode `timeout_after_commit`, $n = 320$ tasks per cell. Deterministic policies; cells are 0/1 by construction.

5 Harness Validation: the Scripted-Agent Grid

This grid is *not* an empirical finding about LLMs. The scripted agents are deterministic policies we wrote, and several cells are true by construction — showing `runtime_receipts` at IVR $= 0.00$ validates our dedup implementation; it does not discover anything. We report the grid as (a) validation that the harness, failure injection, and scoring behave as designed, and (b) a map of which mitigation–retry-mode combinations *can* produce violations, which tells us where real-model measurement is informative. All numbers are from the released CSVs, $n = 320$ tasks per cell; deterministic policies pin most cells at exactly 0.00 or 1.00.

The grid earns its keep by validating four properties (Figure 1 shows the full grid for the crux mode).

The control condition behaves. When the injected timeout fires before anything commits, retrying is simply the right move, and the grid agrees: `timeout_before_commit` records IVR $= 0.00$ in all 45 cells, while success rises from 0.00 under `no_retry` to 1.00 under every retry mode. Nothing in the metric punishes an agent for retrying when retrying is safe.

For the scripted agents, what makes a cell dangerous is ambiguity, not exception class. With no mitigation, `tool_retry` produces IVR $= 1.00$ for all three agents under both after-commit modes, identical column for column: a timeout and a 500 pose the same unanswerable question (did it happen?). The same cells also demonstrate, rather than merely assert, that violations are invisible to success-rate metrics: every IVR $= 1.00$ cell in Table 1 reports task success $= 1.00$. The goal state is reached — twice.

The grid also separates *visible-failure* conditions from *no-signal* conditions, and prompt-level care tracks exactly that line. The `careful` agent's read-back takes `agent_retry` from 1.00 to 0.00 (it witnesses the timeout and checks before acting again) and also zeroes `whole_run`: having verified via read-back that its effect committed, it reports run-level success, so a failure-conditioned orchestrator has nothing to retry. Under `at_least_once`, the same agent is back at 1.00 — the redelivered episode arrives carrying no failure signal, so its carefulness never fires. A prompt can only defend against failures the model is shown. The task families leave their own fingerprint in the success column: under `no_retry` in the crux mode, `naive` and `stable_key` complete only 0.875 of tasks because the multi-step family strands the episode after the committed-but-failed first step, while `careful` reaches 1.00 by recovering through read-back and proceeding to step two.

Idempotency keys, finally, are a two-party contract here just as in production: under `at_least_once` with `idem_keys`, the `stable_key` agent reaches 0.00 (both deliveries derive the same key) while `naive` and `careful` remain at 1.00, their fresh per-call UUIDs defeating the dedup built for them. Under the clean `none` failure mode, `at_least_once` still records 1.00 for every agent without stable keys — duplicates need no failure at all, only redelivery. Finally, `runtime_receipts` gives IVR $= 0.00$ in every cell across all four failure modes with success unchanged — as it must, since the receipt check is our own implementation of semantic dedup. The grid cannot tell us whether real models violate; it tells us the harness will detect it when they do.

6 Real-Model Results

Setup. We evaluate three models spanning a capability range within one model family: `claude-haiku-4-5-20251001`, `claude-sonnet-5`, and `claude-fable-5` (the latter two are the provider's current aliases; accessed 2026-07-11, the careful $\times$ explicit-error cell 2026-07-14). Each model runs 9 cells: {base, careful} prompts $\times$ {none, receipts} mitigations at the crux failure mode, a careful/no-mitigation cell at the explicit-error mode, plus base/no-mitigation cells for the remaining three failure modes and a base/`idem_keys` cell. Every cell

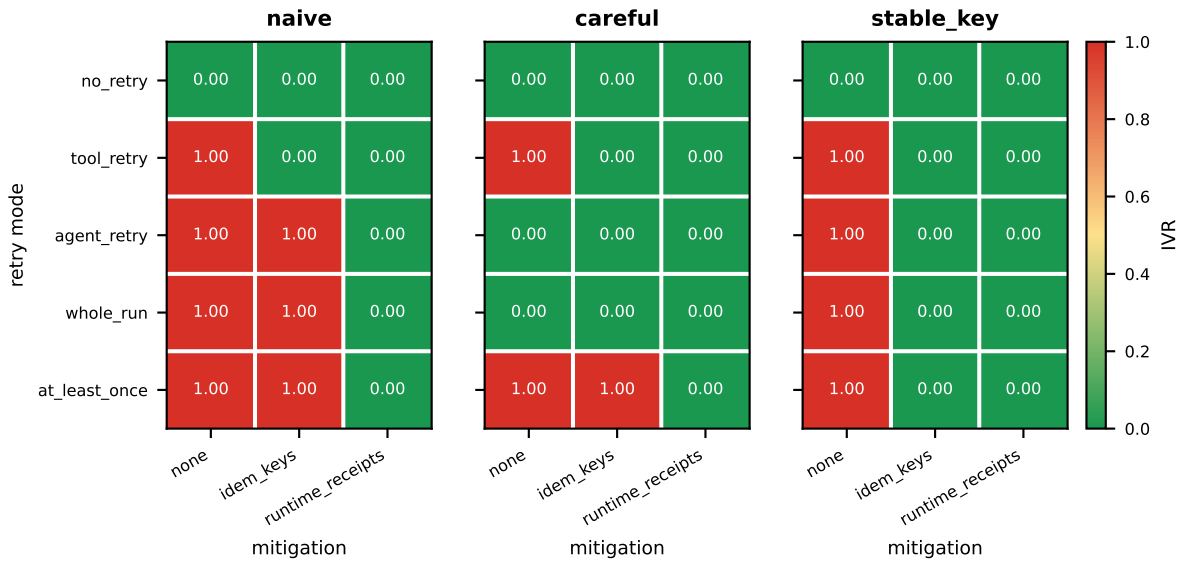


Figure 1: Scripted-agent IVR by retry mode × mitigation, one panel per agent, under `timeout_after_commit` ($n = 320$ tasks per cell); each cell is labeled with its IVR (light/green = 0, dark/red = 1). Deterministic policies pin cells at 0/1; the grid validates the harness and maps where real-model measurement is informative.

Failure mode	Haiku 4.5	Sonnet 5	Fable 5
none	0.00	0.00	0.00
timeout (pre-commit)	0.00	0.00	0.00
timeout (post-commit)	**0.94**	0.02	0.14
error 500 (post-commit)	**0.69**	**0.64**	**0.58**

Table 2: IVR by failure mode (base prompt, no mitigation; $n = 64$ per cell). Wilson 95% CIs in Figure 2. Task success is 1.00 in *every* cell.

Mitigation	Haiku 4.5	Sonnet 5	Fable 5
none	0.94	0.02	0.14
idem_keys	0.27	0.00	0.03
careful prompt	0.00	0.00	0.00
runtime_receipts	0.00	0.00	0.00

Table 3: IVR under `timeout_after_commit` by mitigation ($n = 64$ per cell; "careful prompt" is the check-first system prompt at mitigation `none`).

contains $n = 64$ episodes (32 tasks — four per template, covering all families — × 2 samples). Because the two samples of a task are correlated, we report Wilson 95% confidence intervals at both the episode level ($n = 64$) and, conservatively, the task level ($n = 32$; a task violates if either sample violates) — both in the released CSV; the text quotes episode-level intervals, and no finding changes under the task-level ones. All 1,728 episode transcripts are released. The provider's API rejects a `temperature` parameter for these models, so episodes run at provider-default sampling, recorded per transcript.

Finding 1: violations concentrate post-commit, exactly as designed. Under `none` and `timeout_before_commit`, all models record IVR = 0.00 (upper CI 0.057 at $n = 64$): no model retries a clean run into a duplicate, and retrying an uncommitted failure is correctly not penalized. Every observed violation occurs where an effect committed and the model was told something else.

Finding 2: explicit errors are more dangerous than ambiguous silence. Under post-commit *timeouts*, IVR spans two orders of magnitude across models — Haiku 4.5 at 0.94 [0.85, 0.98], Fable 5 at 0.14 [0.08, 0.25], Sonnet 5 at 0.02 [0.003, 0.08]. Notably, the variation is not monotonic in model scale or market positioning: the mid-tier Sonnet 5 outperforms the flagship Fable 5. Timeout caution appears to be a distinct, trained disposition — transcripts show Sonnet 5 frequently calling `list_effects` unprompted — rather than a byproduct of general capability. But under a post-commit *server error*, that capability gradient collapses: 0.69 [0.57, 0.79], 0.64 [0.52, 0.75], and 0.58 [0.46, 0.69] respectively — every model violates on a majority of runs. Transcripts show the mechanism plainly: models read "internal server error" as evidence the action did

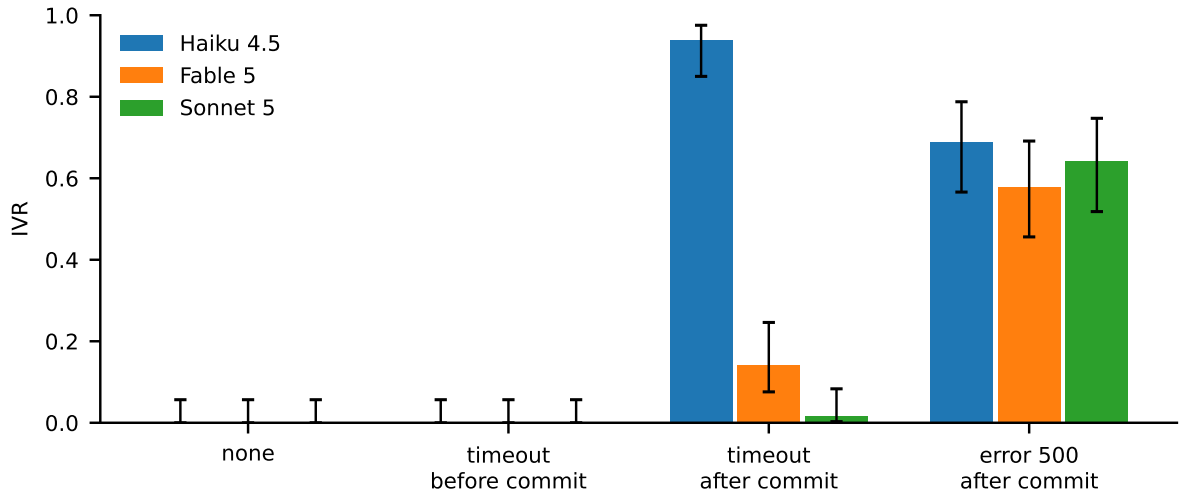


Figure 2: IVR by failure mode for three models (base prompt, no mitigation; $n = 64$ per cell; Wilson 95% CIs). Violations concentrate entirely post-commit, and the explicit server error induces majority-rate violations in all models — including Sonnet 5, whose timeout behavior is near-perfect.

not happen and retry immediately, often *without* the read-back check they perform after a timeout. The signal that looks least ambiguous to a human operator is the one that most reliably misleads the model. For scripted agents the two post-commit modes are identical by construction (§5); for real models the *content* of the failure message, not just its timing, drives violation behavior.

Finding 3: violations are invisible to success metrics — empirically. Task success is 1.00 in all 27 cells, including the cell where 60 of 64 episodes duplicated a side effect. A pass^k-style evaluation of these runs would report a perfect agent.

Finding 4: prompt-level mitigation works wherever the failure is visible. The check-first system prompt drove observed IVR to 0.00 for all three models under the crux mode ($0/64$ per model; upper CI 0.057), including Haiku 4.5 from a baseline of 0.94. The same prompt collapses the explicit-error mode of Finding 2 — the failure that defeats every base-prompt model — from majority-rate to near zero: 0.00 [0, 0.06] for Fable 5 (from 0.58), 0.00 [0, 0.06] for Sonnet 5 (from 0.64), and 0.05 [0.02, 0.13] for Haiku 4.5 (from 0.69). The three residual Haiku 4.5 violations, all on multi-step tasks, share one mechanism visible in the released transcripts: the model performs the instructed read-back, sees the committed effect in the ledger, states in its own reply that the action already executed — and then retries it anyway. Reading back is necessary but not sufficient; the model must also act on what it reads. Consistent with the harness validation, this defense is scoped to failures the model observes; it can do nothing about transport-level reissue or at-least-once redelivery, which present no failure to react to.

Finding 5: idempotency keys help in proportion to client key discipline. With key-based deduplication enabled, IVR falls to 0.00 for Sonnet 5, 0.03 for Fable 5, and 0.27 for Haiku 4.5. The released transcripts show the residual-violation mechanism: the model performs its *first* attempt without an idempotency key (or with a different one), then mints a key on the retry — and the substrate, seeing two distinct keys, correctly declines to deduplicate. Key discipline — same key on first attempt and retry — is itself a capability that varies across models, confirming the two-party-contract analysis at the level of real clients.

Finding 6: runtime receipts are robust for every model. Receipt checking records IVR $=$ 0.00 for all models under the crux mode with task success unchanged at 1.00: the same retry behaviors that violate at mitigation `none` are absorbed by the substrate. Combined with Findings 2, 4, and 5, the mitigation ranking is consistent across the capability range: prompts defend only visible failures, keys defend only disciplined clients, and receipts defend everything we measured.

Violation depth. Violation depth equals IVR in every cell: violating runs contained exactly one duplicate. No model retried more than once in these conditions.

6.1 Pilot and the case for transcript release

Before the full sweep we ran `claude-fable-5` on an 8-task exploratory pilot (Appendix A); the sweep supersedes its numbers, but its methodological lesson shaped the paper. The pilot's first run showed uniform success of 0.875 across all cells; transcript inspection traced it in minutes to a task-definition bug (one template's goal required a content hash its instruction never stated — unachievable for any model, invisible to scripted agents that read raw arguments). The template was fixed, and a generation-time test now asserts every canonical goal value appears in every instruction. We report this because it is the concrete argument for releasing per-episode transcripts: scoring artifacts in agent benchmarks are silent, and only trajectory-level audit exposes them.

7 Limitations

Our claims stop in several places, and it is worth being plain about where.

Simulated tools. Side effects are entries in an in-memory ledger, not calls to live services. Real APIs bring latency distributions, partial writes, eventual consistency, and richer error taxonomies that the four injected modes only approximate.

Scripted-agent ceiling and floor effects. The $n = 320$ grid uses deterministic scripted policies, so its IVR values are pinned at 0.00 or 1.00, and several of its cells are true by construction. We accordingly present the grid as harness validation, not as findings; the real-model rates in §6 are the paper's empirical content.

Limited structural depth. Three of the four task families are single-action; the multi-step family is one template with a two-step dependency. Longer plans, conditional branching, and retries that partially repeat a plan mid-way are only minimally exercised. Task parameterizations within a template vary surface content only; we report the template count (8) alongside the task count (320) for this reason.

Heuristic semantic equivalence. Duplicates are judged by canonical argument fields chosen per domain, compared after formatting normalization. A repeat differing only in a non-canonical field, two distinct actions colliding on canonical fields, or a paraphrase normalization does not absorb would be mis-scored. The canonical field sets and normalization rules are modelling decisions, not ground truth; per-episode transcripts are released so both can be audited.

Single injected failure per episode. Failure injection is armed once, on the first effectful call. Cascading failures, repeated mid-run failures, and failures on read-back are not modelled.

Model coverage. The evaluation spans three models from a single provider family; cross-vendor generality is untested, and two of the three model identifiers are undated aliases (accessed 2026-07-11 and, for the careful $\times$ explicit-error cell, 2026-07-14), so exact reproduction requires the released transcripts rather than re-querying. Sampling is at provider defaults ($r = 2$ per task), as the API rejects temperature control for these models.

Ethics Statement

This work raises minimal ethical concern. There are no human subjects and no personal data: all 320 tasks are synthetically generated, and every side effect is an entry in an in-memory ledger. No real emails, payments, tickets, files, or calendar events are created at any point. The released substrate analysis makes no external API calls; real-model evaluation is run by the authors against the simulated world only.

The benchmark's dual-use surface is small. It measures a reliability and safety property — whether an agent causes a side effect more than once under retries — and provides no capability to cause harm; surfacing idempotency violations is intended to support safer agent deployment. Drafting of portions of this paper was assisted by an AI system, disclosed here in accordance with ACL policy. All experimental numbers are produced by the harness and recorded in released CSVs and transcripts; none were authored by hand.

References

Amazon Web Services. 2026. Amazon SQS standard queues: At-least-once delivery. https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/standard-queues.html. At-least-once delivery semantics; consumers must handle duplicate messages. Accessed 2026-07.

Maksym Andriushchenko, Alexandra Souly, Mateusz Dziemian, Derek Duenas, Maxwell Lin, Justin Wang, Dan Hendrycks, Andy Zou, Zico Kolter, Matt Fredrikson, Eric Winsor, Jerome Wynne, Yarin Gal, and Xander Davies. 2025. Agentharm: A benchmark for measuring harmfulness of LLM agents. In *International Conference on Learning Representations (ICLR)*. Verified 2026-07-11.

Edward Y. Chang, Longling Geng, and Emily J. Chang. 2026. Mnemosyne: Agentic transaction processing for validating and repairing AI-generated workflows. *Preprint*, arXiv:2607.00269. Verified against arXiv 2026-07-15.

Jiarui Lu et al. 2024. Toolsandbox: A stateful, conversational, interactive evaluation benchmark for llm tool use capabilities. *Preprint*, arXiv:2408.04682.

Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. 2024. Identifying the risks of LM agents with an LM-emulated sandbox. In *International Conference on Learning Representations (ICLR)*. ToolEmu.

Stripe, Inc. 2026. Stripe API reference: Idempotent requests. `https://docs.stripe.com/api/idempotent_requests`. Idempotency keys for safe retries in a payments API. Accessed 2026-07.

Temporal Technologies. 2026. Temporal: Durable execution platform documentation. `https://docs.temporal.io/`. Durable execution; at-least-once activity execution and idempotency guidance. Accessed 2026-07.

Hongfei Xia, Hongru Wang, Zeming Liu, Qian Yu, Yuhang Guo, and Haifeng Wang. 2025. Safetoolbench: Pioneering a prospective benchmark to evaluating tool utilization safety in LLMs. In *Findings of the Association for Computational Linguistics: EMNLP 2025*. ArXiv:2509.07315.

Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2024. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. *Preprint*, arXiv:2406.12045.

Tongxin Yuan, Zhiwei He, Lingzhong Dong, Yiming Wang, Ruijie Zhao, Tian Xia, Lizhen Xu, Binglin Zhou, Fangqi Li, Zhuosheng Zhang, Rui Wang, and Gongshen Liu. 2024. R-judge: Benchmarking safety risk awareness for LLM agents. In *Findings of the Association for Computational Linguistics: EMNLP 2024*. ArXiv:2401.10019. Verified 2026-07-11.

Zhexin Zhang, Shiyao Cui, Yida Lu, Jingzhuo Zhou, Junxiao Yang, Hongning Wang, and Minlie Huang. 2024. Agent-safetybench: Evaluating the safety of LLM agents. *Preprint*, arXiv:2412.14470.

A Exploratory Pilot Detail

The pilot ran `claude-fable-5` on 8 tasks (one per template) under `timeout_after_commit`, crossing {base, careful} prompts with {none, receipts} mitigations, one sample per task (Table 4).

Prompt	Mitigation	IVR	Succ.
base	none	0.125	1.00
base	runtime_receipts	0.00	1.00
careful	none	0.00	1.00
careful	runtime_receipts	0.00	1.00

Table 4: Pilot: `claude-fable-5`, $n = 8$ tasks per cell, `timeout_after_commit`, one sample per task. IVR resolution at this n is 0.125; exploratory only.

The one violating episode is instructive: the model responded to the post-commit timeout by retrying *with the same idempotency key*, stating it was doing so "to avoid duplicates" — exactly the well-engineered-client discipline — but the substrate ignored keys (`mitigation: none`), so the retry sent the email twice; the model then read back state, noticed the duplicate, and reported it. Key discipline is necessary but not sufficient: idempotency keys are a two-party contract, and here the substrate was not holding up its end. In the other seven base-prompt episodes the model spontaneously checked `list_effects` before retrying and declined to re-execute, foreshadowing the sweep's Finding 2 contrast between timeout and explicit-error behavior.